

Effective Population Size

John D. Robinson

2026-06-16

Contents

Estimating Effective Population Size (N_e)	1
Create a GENEPOP file (from scratch)	1
Run NeEstimator	5
N_e estimates	6

Estimating Effective Population Size (N_e)

In this markdown, we'll use the linkage disequilibrium (LD) method ([Hill 1981](#)) to estimate effective population size of the three black bear populations in Georgia using the samples processed via capture-seq. For this analysis, we'll use the same set of SNPs (`combined.vcf`) analyzed for genetic diversity (see `2_diversity`) and population differentiation (see `3_structure`). Note that we'll also avoid using intrachromosomal comparisons when calculating r^2 for these estimates ([Waples et al. 2016](#)).

The software `NeEstimator v2` can be downloaded and installed from [this GitHub repository](#).

Create a GENEPOP file (from scratch)

The specifications of this format can be found [here](#).

First, generate a file with genotypes for all samples in the `combined.vcf` file in 012 format...

```
ulimit -n 3000
vcftools --vcf combined.vcf --out combined --012
```

VCFTools - 0.1.17

(C) Adam Auton and Anthony Marcketta 2009

Parameters as interpreted:

```
--vcf combined.vcf
--012
--out combined
```

Warning: Expected at least 2 parts in INFO entry: ID=AF,Number=A,Type=Float,Description="Estimated allele frequency in the sample"

Warning: Expected at least 2 parts in INFO entry: ID=PRO,Number=1,Type=Float,Description="Reference allele frequency in the sample"

Warning: Expected at least 2 parts in INFO entry: ID=PAO,Number=A,Type=Float,Description="Alternate allele frequency in the sample"

Warning: Expected at least 2 parts in INFO entry: ID=SRP,Number=1,Type=Float,Description="Strand balance
Warning: Expected at least 2 parts in INFO entry: ID=SAP,Number=A,Type=Float,Description="Strand balance
Warning: Expected at least 2 parts in INFO entry: ID=AB,Number=A,Type=Float,Description="Allele balance
Warning: Expected at least 2 parts in INFO entry: ID=ABP,Number=A,Type=Float,Description="Allele balance
Warning: Expected at least 2 parts in INFO entry: ID=RPP,Number=A,Type=Float,Description="Read Placement
Warning: Expected at least 2 parts in INFO entry: ID=RPPR,Number=1,Type=Float,Description="Read Placement
Warning: Expected at least 2 parts in INFO entry: ID=EPP,Number=A,Type=Float,Description="End Placement
Warning: Expected at least 2 parts in INFO entry: ID=EPPR,Number=1,Type=Float,Description="End Placement
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=CIGAR,Number=A,Type=String,Description="The extend
After filtering, kept 241 out of 241 Individuals
Writing 012 matrix files ... Done.
After filtering, kept 10717 out of a possible 10717 Sites
Run Time = 0.00 seconds

Then, recreate the population-specific snps objects we used in 19_diversity.

```
#Load metadata file
samps <- read.csv("../input/sampleMetaData_cloneID.csv", header = TRUE)

#Build a popmap file to map individual samples back to their population
popmap <- read.table("combined.012.indv", header = FALSE)
names(popmap) <- "vcfID"
popmap$region <- popmap$ugaID <- popmap$year <- NA
for(i in 1:nrow(popmap)) {
  if(strsplit(popmap$vcfID[i], split="_")[[1]][1] == "UGAh") {
    popmap$region[i] <- samps$region[which(samps$vcfID ==
                                           gsub("h", "", popmap$vcfID[i]))]
    popmap$ugaID[i] <- samps$ugaID[which(samps$vcfID ==
                                         gsub("h", "", popmap$vcfID[i]))]
    popmap$year[i] <- samps$year[which(samps$vcfID ==
                                       gsub("h", "", popmap$vcfID[i]))]
  } else {
    popmap$region[i] <- samps$region[which(samps$vcfID ==
                                           popmap$vcfID[i])]
    popmap$ugaID[i] <- samps$ugaID[which(samps$vcfID == popmap$vcfID[i])]
    popmap$year[i] <- samps$year[which(samps$vcfID == popmap$vcfID[i])]
  }
}

if(popmap$region[i] %in% c("CGA", "CGA-H")) {
  if(strsplit(popmap$vcfID[i], split="_")[[1]][1] == "UGAh") {
    popmap$region[i] <- "CGA12"
  } else {
    popmap$region[i] <- "CGA23"
  }
}
}

snps <- read.table("combined.012", header = FALSE)
dim(snps) #rows are individuals, columns are SNPs
```

```
## [1] 241 10718
```

```
dim(popmap)
```

```
## [1] 241 4
```

```
#Missing data are coded as -1's
```

```
#Create subset datasets for each "population" in our dataset
```

```
# 1. The focal population in central GA sampled in this project - `CGA23`
```

```
# 2. Historic samples from central GA (from ~2012) - `CGA12`
```

```
# 3. The north GA population - `NGA`
```

```
# 4. The south GA population - `SGA`
```

```
#Make CGA12 & CGA23 snp datasets
```

```
CGA12snps <- snps[which(popmap$region == "CGA12"),]
```

```
CGA12pop <- popmap[which(popmap$region == "CGA12"),]
```

```
CGA23snps <- snps[which(popmap$region == "CGA23"),]
```

```
CGA23pop <- popmap[which(popmap$region == "CGA23"),]
```

```
nrow(CGA12snps)
```

```
## [1] 15
```

```
nrow(CGA23snps)
```

```
## [1] 169
```

```
NGAsnps <- snps[popmap$region == "NGA",]
```

```
NGApop <- popmap[popmap$region == "NGA",]
```

```
nrow(NGAsnps)
```

```
## [1] 31
```

```
SGAsnps <- snps[popmap$region == "SGA",]
```

```
SGApop <- popmap[popmap$region == "SGA",]
```

```
nrow(SGAsnps)
```

```
## [1] 26
```

There were four bears with NGA ancestry in our CGA sample. These were offspring of a mother translocated to CGA from NW GA (see `3_structure` for more details). This artificial mixture could bias our estimates of N_e downwards (via addition of mixture LD). Therefore, we'll remove these four individuals from the dataset before proceeding with the estimation.

```
#Remove NGA bears from CGA sample
```

```
dropbears <- which(CGA23pop$sugaID %in% c("74-B", "73-B", "83-RK", "89-RK"))
```

```
length(dropbears)
```

```
## [1] 4
```

```
CGA23pop <- CGA23pop[-dropbears,]
CGA23snps <- CGA23snps[-dropbears,]
dim(CGA23snps)
```

```
## [1] 165 10718
```

```
dim(CGA23pop)
```

```
## [1] 165 4
```

Now create an input file in GENEPOP format.

```
#Create GENEPOP input
loclist <- paste0("SNP", 1:ncol(snps[, -1]))
makeGP <- function(x) {
  x[which(x == "0")] <- "0101"
  x[which(x == "1")] <- "0102"
  x[which(x == "2")] <- "0202"
  x[which(x == "-1")] <- "0000"
  return(x)
}

#For each of the 4 populations...
#CGA 2012
CGA12gp <- apply(CGA12snps[, -1], 2, FUN=makeGP)
CGA12gp <- cbind(paste0(CGA12pop$ugaID, ","), CGA12gp)

#CGA 2023
CGA23gp <- apply(CGA23snps[, -1], 2, FUN=makeGP)
CGA23gp <- cbind(paste0(CGA23pop$ugaID, ","), CGA23gp)

#NGA
NGAgp <- apply(NGAsnps[, -1], 2, FUN=makeGP)
NGAgp <- cbind(paste0(NGApop$ugaID, ","), NGAgp)

#SGA
SGAgp <- apply(SGAsnps[, -1], 2, FUN=makeGP)
SGAgp <- cbind(paste0(SGApop$ugaID, ","), SGAgp)

cat("NeEstimator - all pops", file = "CGA_GPfile.txt")
cat("\n", file = "CGA_GPfile.txt", append = T)
cat(paste0(loclist, collapse = ","), file = "CGA_GPfile.txt", append = T)
cat("\n", file = "CGA_GPfile.txt", append = T)
cat("POP", file = "CGA_GPfile.txt", append = T)
cat("\n", file = "CGA_GPfile.txt", append = T)
write.table(CGA12gp, file = "CGA_GPfile.txt",
  append = T, quote = F, sep = " ", row.names = F, col.names = F)
cat("POP", file = "CGA_GPfile.txt", append = T)
cat("\n", file = "CGA_GPfile.txt", append = T)
write.table(CGA23gp, file = "CGA_GPfile.txt",
  append = T, quote = F, sep = " ", row.names = F, col.names = F)
```

```

cat("POP", file = "CGA_GPfile.txt", append = T)
cat("\n", file = "CGA_GPfile.txt", append = T)
write.table(NGAgp, file = "CGA_GPfile.txt",
            append = T, quote = F, sep = " ", row.names = F, col.names = F)
cat("POP", file = "CGA_GPfile.txt", append = T)
cat("\n", file = "CGA_GPfile.txt", append = T)
write.table(SGAgp, file = "CGA_GPfile.txt",
            append = T, quote = F, sep = " ", row.names = F, col.names = F)

#Now create the SNP info file with chromosome information
cpos <- read.table("combined.012.pos", header = FALSE)
snpkey <- data.frame(CHROM=cpos[,1], SNP=loclist)
write.table(snpkey, "SNPkey.txt",
            quote = FALSE, row.names = FALSE, col.names = TRUE)

```

Run NeEstimator

The analysis requires info and option files as well. These were created in a text editor.

```

cat ../input/CGA_info.txt

cat ../input/CGA_option.txt

```

```

1                                     * A number = sum of method(s) to run: LD(=1), H
/scratch/jdrobins/CGA_PopGen/popgen/    * Input Directory
CGA_GPfile.txt                          * Input file name
2                                     * 1 = FSTAT format, 2 = GENEPop format
/scratch/jdrobins/CGA_PopGen/popgen/    * Output Directory
CGA_Ne.txt                              * Output file name (put asterisk adjacent to the na
2                                     * Number of critical values, added 1 if a run by
0.05                                    * Critical values, a special value '1' is for r
0                                     * 0: Random mating, 1: Monogamy (LD method)
1 0 1 1 * First number = sum of method(s) to have extra output: LD(=1), Het(=2), Coan(=4), Temporal
0                                     * Maximum individuals/pop. If 0: no limit
0                                     * First entry n1 = 0: No Freq output. If n1 = -1: Freq. output up to population 50. Two ent
0                                     * For Burrow output file (up to 50 populations can have output). See remark below
1                                     * Parameter CI: 1 for Yes, 0 for No
1                                     * Jackknife CI: 1 for Yes, 0 for No
0                                     * Up to population, or range of populations to run (if 2 entries). If first entry = 0: no r
0                                     * All loci are accepted
0                                     * Enter 1: A file is created to document missing data if there are any in input file. Enter
2 SNPkey.txt * Line for chromosomes/loci option and file

```

Now run NeEstimator with these files, this doesn't take too terribly long...

n.b., this chunk did not work on the UGA cluster. Repeated errors that NeEstimator exited with status 134. However, the program generates the expected output. Setting `eval=FALSE` in this chunk header and using output from the first submission (earlier today) in the generation of the markdown.

```
NeEst="/home/jdrobins/Software/NeEstimator2.X/Ne2L"
$NeEst i:../input/CGA_info.txt o:../input/CGA_option.txt
```

N_e estimates

Now let's look at the output...

```
cat CGA_NexLD.txt
```

```
Output from NeEstimator v.2
Starting time: Tue Jun 16 08:56:44 2026
Input File: "CGA_GPfile.txt"
```

```
Number of Loci = 10717
```

```
LINKAGE DISEQUILIBRIUM METHOD, Mating Model: Random
```

```
Lowest allele frequency used: 0.0500
```

```
Up to 17 rightmost characters are shown for Population name.
```

Population [#:Name]	Samp Size	Weighted Mean	Ind. Alleles	r^2	$\text{Exp}(r^2)$	N_e	Parametric CI		Jackknife CI		(Eff.df)
							Low	High	Low	High	
1:336-23	15	8.9	1502017	0.146174	0.142804	89.7	81.5	99.6	4.2	Infinite	146
2:336-23	165	118.6	904032	0.015393	0.008492	46.1	45.8	46.5	38.5	55.8	1232
3:80-NGA	31	25.7	3035676	0.044377	0.042415	155.3	149.8	161.1	54.6	Infinite	1385
4:42-SGA	26	20.2	2993825	0.053398	0.052979	733.1	608.4	921.2	64.8	Infinite	1388

```
Ending time: Tue Jun 16 09:04:37 2026
```